MD3135 Data Science — Beginner-Friendly 12-Week Roadmap

Written for someone starting from zero in Data Science/Statistics — every technical term gets a plain-English explanation before the formal one.

Read this first: how to use this roadmap

You don't need to feel behind. Nothing here requires prior Data Science knowledge — only basic Python syntax (if/else, loops, functions), which you already have.

Every week follows the same 5-day rhythm so you're never guessing what to do:

Day	What you do
Day 1	**Understand** — read/watch the concept in plain English before touching code
Day 2	**Rewrite in your own words** — 1 paragraph per concept, no copy-paste
Day 3	**Code it** — small, ugly, working code is the goal, not clean code
Day 4	**Practice questions** — VIT-style short answers + 1 numerical
Day 5	**Apply to a real dataset** — even 10 minutes counts
Weekend	Sat = tutorial/mini-project, Sun = revise the week in 15 min

If a week feels heavy, it's fine to spend an extra 2–3 days on it and borrow time from Week 10 (marked optional/bonus below).

The Big Picture (why each phase exists)

```
Phase 1 (Wk 1-2):  What IS data, and how do we describe it?
Phase 2 (Wk 3-4):  How do we hold and manipulate data in code?
Phase 3 (Wk 5):    How do we fix messy, real-world data?
Phase 4 (Wk 6-7):  How do we explore data and SEE patterns?
Phase 5 (Wk 8):    How do we prepare data for a model?
Phase 6 (Wk 9):    How do we predict a number from data?
Phase 7 (Wk 10):   (Bonus) How do we predict a category?
Phase 8 (Wk 11):   How do we know if our model is actually good?
Phase 9 (Wk 12):   Put it all together + revise for the exam
```

Notice the story: each phase is a *prerequisite* for the next one. You can't clean data (Phase 3) without knowing what a data type is (Phase 1). You can't build a model (Phase 6) without clean, explored data (Phases 3–4). This is why the order matters — don't skip ahead even if a later topic sounds more exciting.

Week 1 — Unit I: What is Data Science?

Plain-English framing: Data Science is the skill of turning raw information (numbers, text, logs) into decisions. "Statistical inference" just means: making a reasonable guess about a big group by looking at a small piece of it. Example: you taste one spoon of soup (sample) to judge the whole pot (population) — you don't need to drink it all.

Learn

- What is Data Science, and how is it different from just "statistics"? (Data Science adds programming + real messy data + decision-making)
- **Big Data** = data too large/fast/varied for a normal spreadsheet to handle
- **Datafication** = the trend of turning everything (friendships, driving habits, sleep) into measurable data
- **Population vs Sample** = the whole group vs. the small piece you actually study

- **Overfitting** (plain terms): imagine memorizing exam answers instead of understanding the topic — you'll ace that exact paper but fail a slightly different one. A model that "overfits" memorizes the training data instead of learning the pattern.
- Python environment setup — make sure Jupyter/Colab actually runs before Week 2.

Python (refresh only, don't relearn from scratch)

`variables, if/else, for/while loops, functions, lists, dictionaries, tuples, sets, file handling`

Tutorial 1

Simulate Population vs Sample — e.g., take a list of 100 numbers, then repeatedly grab random groups of 10 and compare their average to the whole list's average.

Mini practical

Load a CSV using plain Python (no Pandas yet) — read it with `open()` or `csv` module, loop through rows, count something simple (e.g., how many rows match a condition) using only lists/dictionaries.

Exam-ready by Friday

- Define: Data Science, Datafication, Overfitting, Statistical Inference
- Differentiate: Population vs Sample
- One-line explanation: why do we use samples instead of populations?

Week 2 — Unit II: Describing Data With Numbers

Plain-English framing: Before you can analyze anything, you need a vocabulary for "what kind of data is this?" and "how do I summarize a pile of numbers into a couple of useful numbers?"

Data types (with beginner examples)

Type	Meaning	Example
Nominal	Categories, no order	Colors: red, blue, green
Ordinal	Categories, WITH order	Ratings: bad, okay, good
Binary	Only 2 options	Yes/No, 0/1
Numeric — Discrete	Countable whole numbers	Number of siblings
Numeric — Continuous	Can be any value in a range	Height, temperature
Asymmetric	Only one value really matters (rest is "absence")	"Has disease" — most people don't, so "yes" carries more info than "no"

Statistics — the two questions every dataset answers

1. **"Where's the center?"** → Mean (average), Median (middle value), Mode (most frequent)
2. **"How spread out is it?"** → Range, Quartiles, Variance, Standard Deviation, IQR

Plain-English gotcha: Mean gets pulled around by extreme values (one billionaire changes the "average" income of a room); Median doesn't. This single sentence answers a very common exam question.

Tutorial 2

Pick 10 numbers. Calculate mean/median/mode/variance/SD **by hand first**, then check with:

```
python
```

```
import numpy as np
np.mean(x); np.median(x); np.std(x); np.var(x)
```

Doing it by hand once is what makes the Python version actually make sense.

Tutorial 3

Take 20–30 real-world examples (age, blood type, movie rating, zip code, temperature…) and classify each as nominal/ordinal/binary/discrete/continuous. This drill alone covers a very common exam question type.

Exam-ready by Friday

Mean vs Median vs Mode · Variance vs SD vs IQR · Discrete vs Continuous · Nominal vs Ordinal · Symmetric vs Asymmetric attributes

---

Week 3 — Unit IV (Part 1): NumPy + PCA (first pass)

Plain-English framing: NumPy is just Python's way of doing fast math on lists of numbers ("arrays") instead of looping manually. A **vector** is a 1D list of numbers, a **matrix** is a 2D grid of numbers (rows × columns).

NumPy — build up slowly

python

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])   # this is a vector
arr[0]        # first element
arr[1:5]      # a slice
arr.shape     # how many elements
```

Then move to 2D:

```python
m = np.array([[1,2,3],[4,5,6]])   # this is a matrix (2 rows, 3 cols)
m[:, 2]        # all rows, column index 2
m[1:4, 2:5]    # a rectangular chunk
m.reshape(3,2) # rearrange dimensions
```

Go slow here — this week's ideas (indexing, slicing, shape) are the foundation for everything in Weeks 4–8.

PCA — first pass, concept only, no math yet

Plain-English: imagine a dataset with 50 columns. Most of that information is redundant or noisy. PCA finds a smaller number of new "super-columns" that still capture most of the important pattern. Think of it as compressing a photo — you lose a little detail but keep the picture recognizable, using far less data.

You do NOT need the linear algebra behind PCA yet. Just be able to answer: *what problem does PCA solve, and why would you want fewer columns?*

Practical

CSV → pick 2–3 numeric columns → convert to NumPy array → compute mean/std on them.

Exam-ready by Friday

Vector vs Matrix · Indexing vs Slicing · What PCA is for (in your own words, no formulas needed yet)

Week 4 — Unit IV (Part 2): Pandas + Lists/Dictionaries for real data

Plain-English framing: Pandas is a table (like Excel) inside Python. A "DataFrame" is just that table. This week is heavy — split it across the full week, don't rush.

Pandas fundamentals — in order of how you'd actually use them

```python
import pandas as pd
df = pd.read_csv("data.csv")   # load
df.head()      # first 5 rows — always look at this first
df.info()      # column names + types
df.describe()  # quick stats on numeric columns
```

Then selecting/filtering:

```python
df['column_name']            # one column
df[df['age'] > 30]           # filter rows
df.loc[0:5, 'age']           # loc = label-based selection
df.iloc[0:5, 2]              # iloc = position-based (numbers only)
```

Memory trick: `loc` = **lo**okup by la**bel** (names). `iloc` = **i**nteger-based **loc**ation (numbers).

Official syllabus requirement — don't skip this

Do one CSV analysis task using **only** lists and dictionaries (no Pandas). Example: read a CSV of students, and using a list of dictionaries, find the average marks per subject. This is explicitly examinable and easy to forget once Pandas feels easier.

Tutorials

Tutorial 5: Vectors, Matrices, Arrays, DataFrames — redo Week 3's NumPy exercises but now compare the same operations in Pandas. **Tutorial 6:** Lists, Functions, Control Structures — build small reusable functions (e.g., a function that returns the average of a list).

Exam-ready by Friday

`loc` vs `iloc` with a working example · what a DataFrame is · why lists/dicts can also represent tabular data

---

Week 5 — Unit III: Cleaning Messy, Real-World Data

Plain-English framing: Real data is never clean. Someone will have typed "N/A", someone will have duplicated a row by accident, someone's age will say "250." This week teaches you to catch and fix those problems.

Data quality — 5 questions to ask of any dataset

1. **Accurate?** — is the data actually correct
2. **Complete?** — are values missing
3. **Consistent?** — does "USA" also appear as "U.S.A" and "United States" elsewhere
4. **Valid?** — does age = -5 make sense
5. **Timely?** — is this data still relevant/current

Missing values

python

```
df.isnull().sum()     # how many missing values per column
df.dropna()            # remove rows with missing values
df.fillna(df.mean())   # fill missing values with the average
```

Duplicates

python

```
df.duplicated()
df.drop_duplicates()
```

Outliers — plain-English test

A value is a likely outlier if it's *far* from the rest. Two common rules:

- **IQR rule:** anything below Q1 − 1.5×IQR or above Q3 + 1.5×IQR
- **Z-score rule:** anything more than ~3 standard deviations from the mean

Data transformation — why bother?

- **Normalization/Standardization:** many models get confused if one column is "age" (0–100) and another is "salary" (0–1,000,000) — scaling puts everything on a comparable footing.
- **Encoding:** models need numbers, not text, so "red/blue/green" gets converted to numbers.

Important, easy-to-miss topic: Hypothesis Testing vs EDA

Plain-English: **EDA** = poking around data with no fixed question, just looking for patterns ("let's see what's interesting here"). **Hypothesis Testing** = you already have a specific claim and you're testing whether the data supports it ("is the average income actually different between these two groups?"). EDA comes first; hypothesis testing comes after you have a specific question to check.

Tutorial 4

Take a deliberately messy CSV (or make one) with missing values, duplicates, and one obvious outlier. Clean it start to finish.

Exam-ready by Friday

Data cleaning steps · outlier detection methods · Hypothesis Testing vs EDA · Normalization vs Standardization

---

Week 6 — Unit V (Part 1): Exploring Data (EDA)

Plain-English framing: EDA is detective work — you're looking for the story the data is telling before you build anything predictive.

Pandas grouping tools

```python
df.groupby('category')['value'].mean()   # average value per category
df.agg(['mean', 'sum', 'count'])
pd.merge(df1, df2, on='id')              # combine two tables
pd.concat([df1, df2])                     # stack two tables
```

EDA in 4 steps

1. **Descriptive** — what are the basic stats? (`describe()`)
2. **Distribution** — is the data spread evenly, or bunched up/skewed?
3. **Correlation** — do two columns move together? (`df.corr()`)
4. **Trends/patterns** — is there a pattern over time or category?

Mini Project #1

Pick a real dataset (Kaggle is fine). Do this exact sequence: load → inspect → clean → stats → outliers → correlation → EDA. Write 5–10 plain-English observations — this matters more than making a graph look nice. Example of a good observation: *"The median salary is lower than the mean, which suggests a right-skewed distribution with a few very high earners."*

Exam-ready by Friday

The EDA workflow, step by step · correlation ≠ causation (know why) · `merge()` vs `concat()`

Week 7 — Unit VI: Making Data Visible (Charts)

Plain-English framing: A chart is just a translation of numbers into a shape your brain can read faster than a table.

Which chart, when? (This table alone answers a common exam question)

Chart	Use it when you want to...
Bar chart	Compare sizes across categories
Histogram	See how one numeric variable is distributed
Box plot	See spread + spot outliers at a glance
Line graph	Show change over time
Scatter plot	See if two variables are related
Pie chart	Show simple proportions of a whole (use sparingly)

Code

python

```python
import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(df['age'])
sns.boxplot(x=df['salary'])
sns.scatterplot(x=df['age'], y=df['salary'])
sns.heatmap(df.corr(), annot=True)
```

Tutorial 7

Recreate all 6 chart types above using one real dataset — you should be able to justify *why* you picked each chart type for each column.

Exam-ready by Friday

Bar vs Histogram (when to use which — this is asked often) · what a box plot shows · when scatter plots are useful

Week 8 — Unit V (Part 2): Preparing Data for a Model

Plain-English framing: Models are picky eaters — they need numeric, appropriately scaled, relevant columns. This week is about getting data "model-ready."

Feature selection & extraction

- **Selection** = choosing which existing columns to keep
- **Extraction** = creating new, more useful columns from existing ones

Encoding (turning categories into numbers)

```python
pd.get_dummies(df['color'])           # One-Hot Encoding
df['size'].map({'S':0,'M':1,'L':2})   # Ordinal Encoding
```

Scaling

```python
from sklearn.preprocessing import StandardScaler, MinMaxScaler
StandardScaler().fit_transform(df[['age','salary']])
```

Plain-English difference: StandardScaler centers data around 0 with a "typical" spread of 1 (good default). MinMaxScaler squeezes everything into a fixed 0–1 range (useful when you need bounded values).

PCA — second pass, now with more depth

Now that you understand scaling, revisit PCA: you always scale data *before* applying PCA, because PCA is sensitive to differences in column scale — a column measured in the thousands would unfairly dominate a column measured in single digits otherwise.

Exam-ready by Friday

StandardScaler vs MinMaxScaler (and why order matters with PCA) · Selection vs Extraction · One-Hot vs Ordinal Encoding

---

Week 9 — Unit VI: Predicting a Number (Regression)

Plain-English framing: Regression answers: "given what I know (hours studied), what number should I expect (exam score)?" It draws the best-fitting straight line through your data points.

Core vocabulary, explained simply

- **Independent variable (X)** = what you already know (hours studied)
- **Dependent variable (Y)** = what you're predicting (exam score)
- **Slope** = for every +1 unit of X, how much does Y change
- **Intercept** = the predicted Y when X = 0
- **Residual** = the gap between your prediction and the real value — smaller is better

Multiple Linear Regression

Same idea, but with several inputs at once: hours studied + attendance + previous marks → predicted exam score.

How good is the model? (Metrics, in plain English)

- **MAE** — average size of your prediction errors (easy to interpret)
- **MSE** — same idea but squares the errors (punishes big mistakes more)
- **RMSE** — square root of MSE (brings it back to the original units)
- **R^2** — "how much of the variation in Y does my model explain?" (0 = none, 1 = perfect)

Code workflow

```
python
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression().fit(X_train, y_train)
preds = model.predict(X_test)
```

Tutorial 8

Linear Regression and Predictive Analytics — use the "hours studied → exam score" example end to end.

Exam-ready by Friday

Regression numericals (practice at least 3) · interpret slope/intercept in words · define MAE/MSE/RMSE/R^2

Week 10 — Bonus (Only If You're On Track): Predicting a Category

Not in the official syllabus — skip entirely if Weeks 1–9 need more time.

Plain-English idea: regression predicts a *number*, classification predicts a *category* (spam/not spam, disease/no disease).

For each algorithm below, only learn: (1) what problem it solves, (2) roughly how it works, (3) when you'd use it, (4) one advantage, (5) one limitation.

- Logistic Regression
- KNN (K-Nearest Neighbors — "classify based on what your closest neighbors are")
- Decision Tree ("a flowchart of yes/no questions")
- Random Forest ("many decision trees voting together")

This is useful for your future ML/AI courses, not for MD3135 marks.

Week 11 — Unit V (Part 3): Is the Model Actually Good?

Plain-English framing: A model can look great on the data it was trained on and still fail on new data — this week is about honest testing.

Train / Validation / Test — why three groups, not one?

- **Training set** — the model learns from this
- **Validation set** — you tune settings using this (without touching the test set)
- **Test set** — final, untouched check of real performance

Cross-Validation

Instead of one lucky/unlucky train-test split, you split the data into K chunks ("folds"), train on K−1 of them, test on the remaining one, and repeat K times, rotating which chunk is held out. This gives a much more reliable performance estimate.

Confusion Matrix — the exam favorite

```
                Actual
              0       1
Predicted 0   TN      FN
Predicted 1   FP      TP
```

- **Accuracy** = (TP+TN) / Total — "how often was I right overall?"
- **Precision** = TP / (TP+FP) — "when I said yes, how often was I right?"
- **Recall** = TP / (TP+FN) — "of all actual yeses, how many did I catch?"
- **F1** = balance of precision and recall (bonus, not core syllabus)

Practice this by hand with made-up numbers (e.g., TP=40, FP=10, FN=5, TN=45) until you can compute all four without looking anything up — this is a very common numerical question.

Exam-ready by Friday

Train/Validation/Test purpose of each · why cross-validation beats a single split · manual confusion matrix calculations

---

Week 12 — Final Project + Full Revision

Track A: One complete project (pick ONE, don't split effort)

- Student Performance Analysis & Prediction
- House Price Analysis & Prediction
- Hospital Patient Analysis & Prediction

Full pipeline: load → inspect → clean → EDA → visualize → engineer features → scale/encode → train/test split → regression → evaluate → write 5+ plain-English insights.

Track B: Revision notebook, one section per unit

For every unit, write out:

1. **Definitions** — one plain-English sentence each
2. **Key differences** (this list alone covers a huge share of MD3135 short-answer questions):
   - Population vs Sample
   - Mean vs Median
   - Normalization vs Standardization
   - `loc` vs `iloc`
   - Training vs Testing
   - Precision vs Recall
   - Hypothesis Testing vs EDA
3. **Programs you can write from memory:** NumPy array ops, matrix ops, CSV via lists/dicts, Pandas DataFrame ops, Linear Regression workflow

4. **Numericals you can solve without a calculator hint:** Mean/Median/Mode, Variance/SD/IQR, Correlation, Regression slope, Confusion Matrix metrics
5. **Diagrams you can draw from memory:** Data Science lifecycle, box plot, histogram, regression line, confusion matrix, PCA concept (before/after compression)

Exam Priority Tiers

Tier	Meaning	Topics
🔴 Must be perfect	Statistics, Python, NumPy, Pandas, Data Cleaning, EDA, Feature Engineering, Linear/Multiple Regression, Train/Test, Cross-Validation, Model Evaluation	
🟠 Must understand	Data Types, PCA, Visualization, Probability distributions	
🟡 Know conceptually	Logistic Regression, KNN, Decision Tree, Random Forest (bonus, not examined)	
○ Ignore this semester	Deep Learning, PyTorch, TensorFlow, CNN, RNN, Transformers, LLMs, MLOps	

A Beginner's Glossary (keep this open in a tab)

Term	Plain-English meaning
Dataset	A table of information
Feature / Column / Attribute / Variable	Same thing — one piece of info about each row
Row / Record / Sample	One entity in your table (one person, one transaction)
Model	A trained pattern-finder that makes predictions
Training	Showing the model examples so it can learn a pattern
Overfitting	Model memorized the training data instead of learning the general pattern
Label / Target	The thing you're trying to predict
Numeric vs Categorical	Numbers you can do math on vs. groups/labels

What NOT to worry about yet

Deep Learning, PyTorch, TensorFlow, CNNs, RNNs, Transformers, LLMs, LangChain, MLOps, advanced NLP/CV. None of it is tested in MD3135, and trying to learn it now will only slow down the fundamentals you actually need first.

SAS (optional, supplementary only — never let it compete with the schedule above)

Use only if you have genuinely free access, and only for reinforcing statistics intuition:

1. Weeks 1–2: **Data Literacy Essentials** / **Introduction to Statistical Concepts**

2. Week 4: **Data Literacy in Practice**
3. Weeks 6–7: **Explore and Visualize: Getting Started**
4. Weeks 8–9: **SAS Analytics: Getting Started**
5. Weeks 10–12: skip unless it's clearly free and you have spare time

Drop SAS entirely during the final 2 weeks before your exam — VIT syllabus takes 70% of your time then, Python practice takes the other 30%.